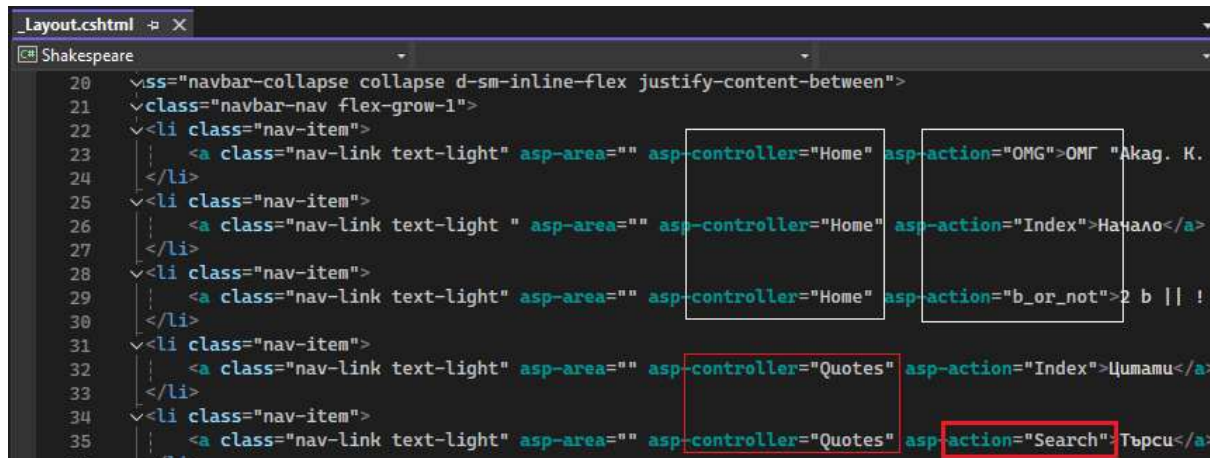


Търсачка

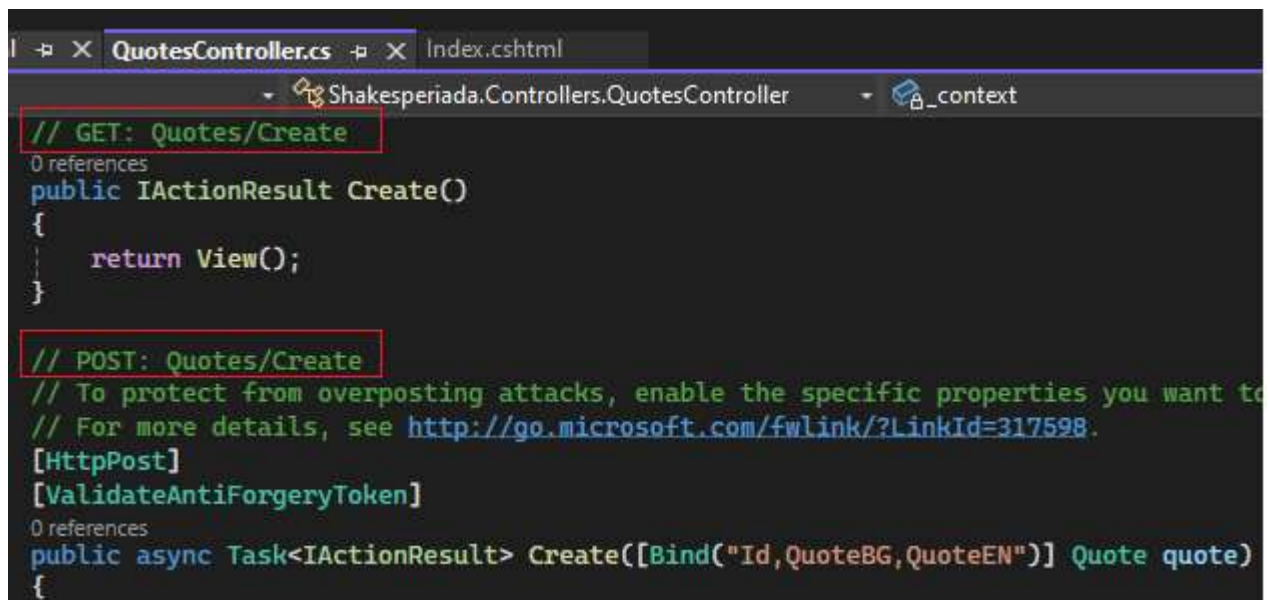
Нека продължим с търсачката, която сме заявили, но още не съществува:



```
20 <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
21 <div class="navbar-nav flex-grow-1">
22 <li class="nav-item">
23 <a class="nav-link text-light" asp-area="" asp-controller="Home" asp-action="OMG">ОМГ "Акаг. К.
24 </li>
25 <li class="nav-item">
26 <a class="nav-link text-light" asp-area="" asp-controller="Home" asp-action="Index">Начало</a>
27 </li>
28 <li class="nav-item">
29 <a class="nav-link text-light" asp-area="" asp-controller="Home" asp-action="b_or_not">2 b || !
30 </li>
31 <li class="nav-item">
32 <a class="nav-link text-light" asp-area="" asp-controller="Quotes" asp-action="Index">Цумаму</a>
33 </li>
34 <li class="nav-item">
35 <a class="nav-link text-light" asp-area="" asp-controller="Quotes" asp-action="Search">Търси</a>
```

Т.е. ние сме декларирали такава страница, но тя още не съществува.

Отиваме в QuotesController, за да създадем събитията за нея. Забелязваме, че има 2 метода: GET и POST за приемане и предаване на данни.



```
// GET: Quotes/Create
0 references
public IActionResult Create()
{
    return View();
}

// POST: Quotes/Create
// To protect from overposting attacks, enable the specific properties you want to bind to
// For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public async Task<IActionResult> Create([Bind("Id,QuoteBG,QuoteEN")] Quote quote)
{
```

Добавяме метод GET за основната страница на търсачката, т.е. казваме да върне изглед View с името Search.

```
QuotesController.cs  X Index.cshtml
Shakespeareana.Controllers.QuotesController

// GET: Quotes /index
3 references
public async Task<IActionResult> Index()
{
    return View(await _context.Quote.ToListAsync());
}

// GET: Search /index
0 references
public IActionResult Search()
{
    return View("Search");
}
```

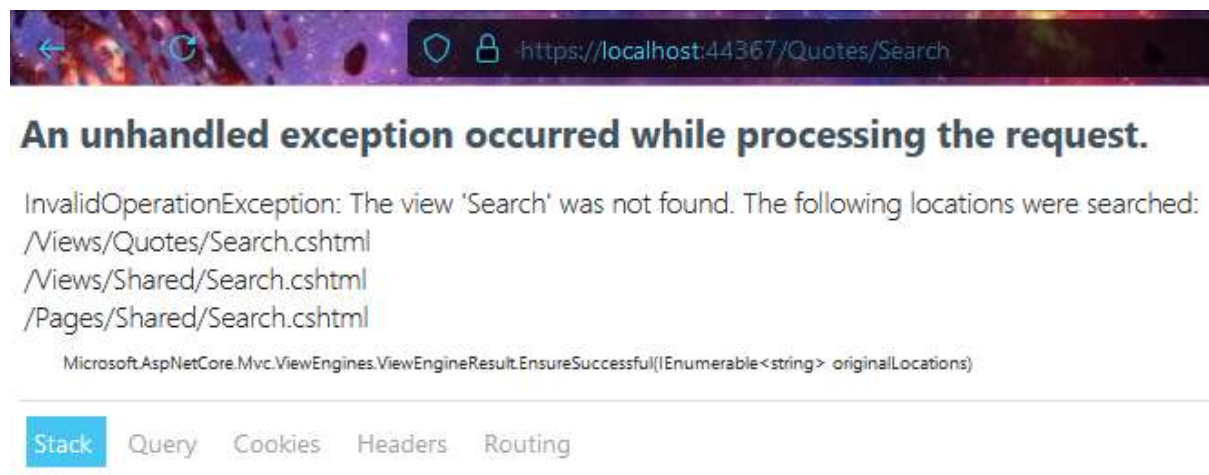
добавяме

Можем и да тестваме, че при изписване на някакъв текст в кавичките, той няма да бъде разпознат като команда, освен, ако няма пълно съвпадение (например „Searching“, „SearchForm“ и т.н.). Може да остане празно return View();

Когато кликнем върху търсачката,

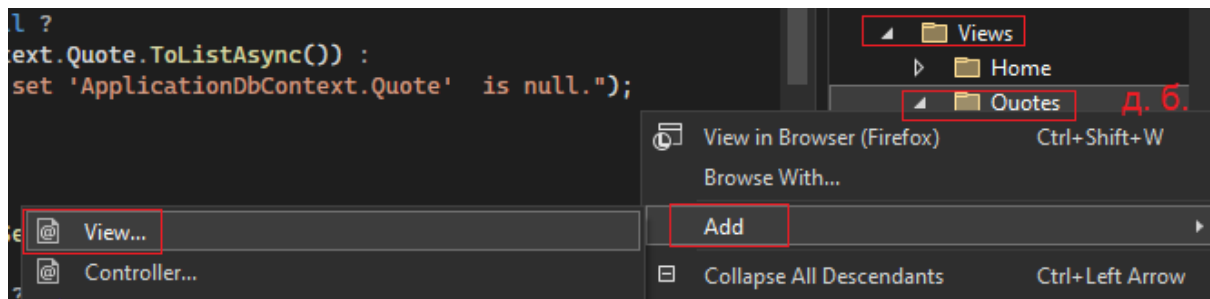


търси тази страница в няколко служебни директории подред, но не я намира. Нека прочетем внимателно кои са тези директории и защо търси именно в тях:

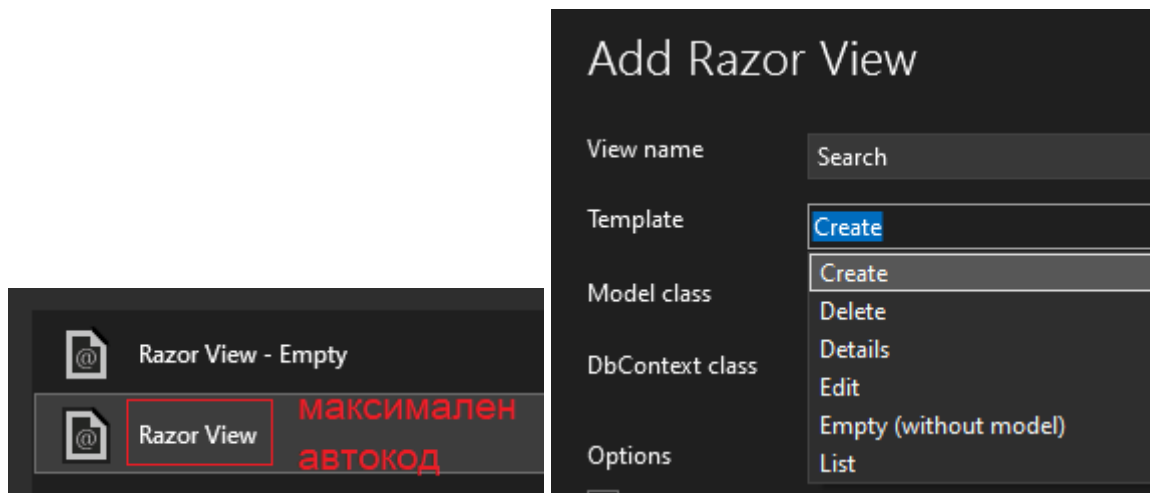


Съответно трябва да си създадем самата страница / изглед на правилното място и с правилното име, като най-добре той да бъде в съответствие с дизайна на останалите страници и да използваме готов шаблон / изрезка Razor (ножица), за да си спестим малко писане с автокод:

Views/Quotes -> д.б. -> Add -> View



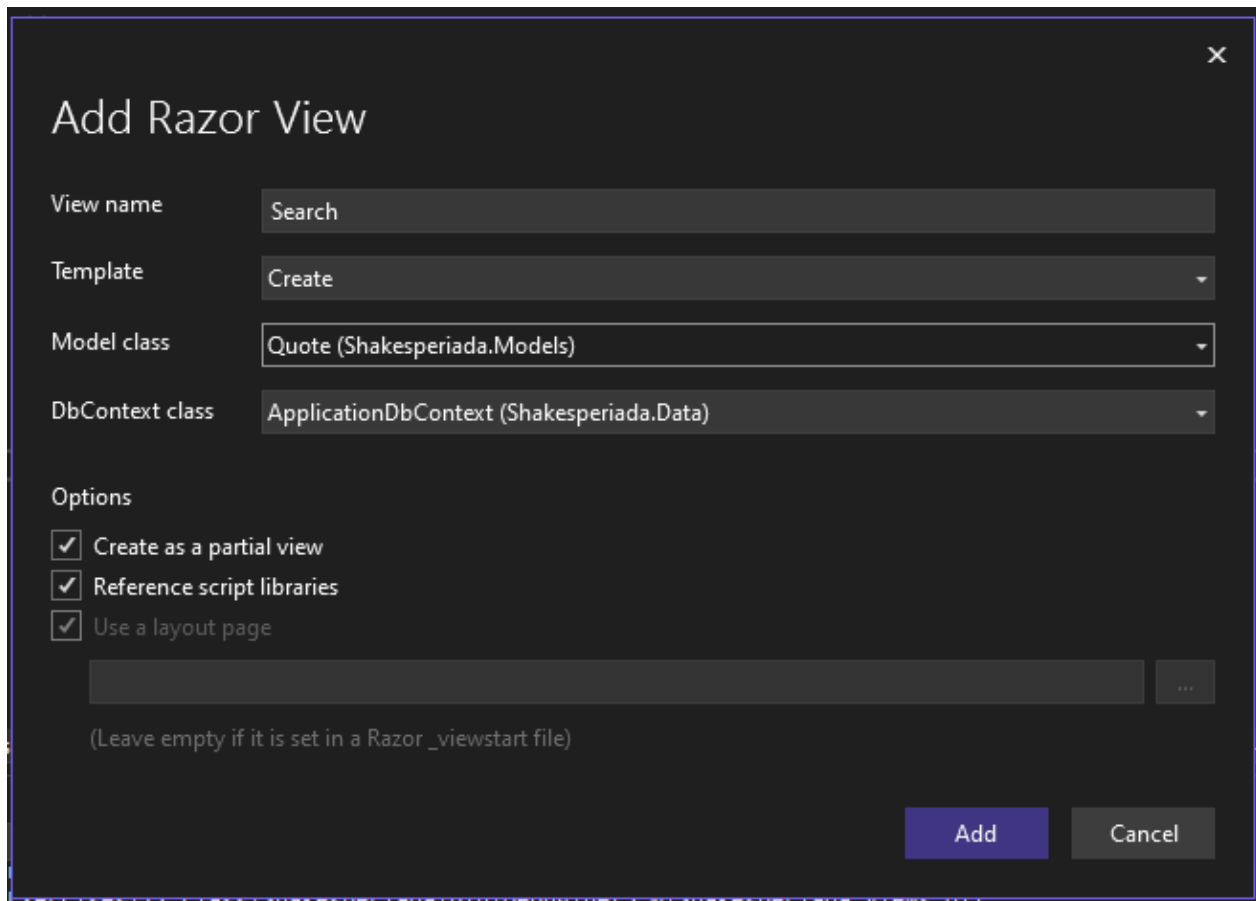
За максимално генериране на код, разбира се, няма да изберем Empty в този проект.



Задаваме правилното име на файла – ще трябва да бъде Search.cshtml, съответно името на самия View е Search.

Template – шаблонът за търсачката трябва да се доближава възможно най-много до някой от предложените. В случая мислим: една търсачка на кое от действията наподобява най-много? На Create (разбира се, ще се редактира и той).

Много е важно и да изберем правилния модел (логиката). Във VS 2022 това е много улеснено спрямо 2016 версията. Това са правилните настройки, като не забравяме да сложим и отметка на Create as a partial view – т.е., че нашият код ще бъде част от другия код на проекта.

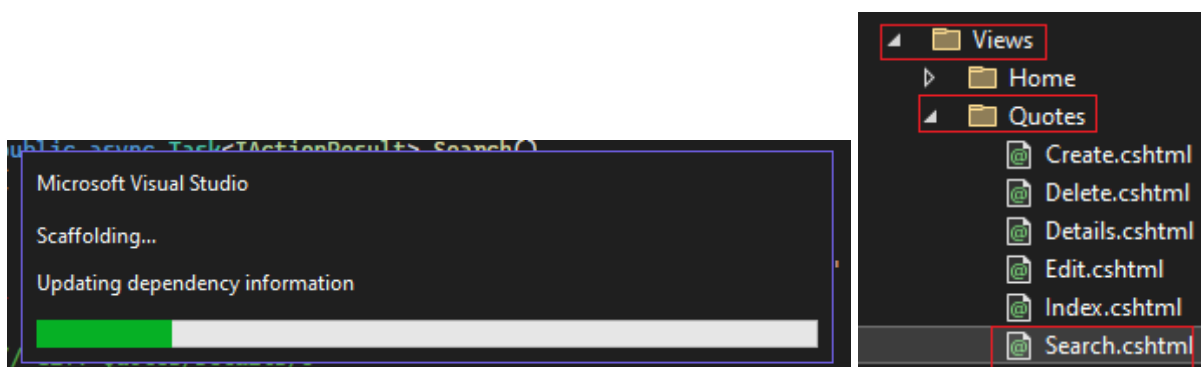


The 'Add Razor View' dialog box is shown with the following settings:

- View name: Search
- Template: Create
- Model class: Quote (Shakesperiada.Models)
- DbContext class: ApplicationDbContext (Shakesperiada.Data)
- Options:
 - ☒ Create as a partial view
 - ☒ Reference script libraries
 - ☒ Use a layout page

Buttons: Add, Cancel

Отново изчакваме бавното скафолдване, което ще генерира много неща вместо нас, а като приключи, нашият изглед Search.cshtml се появява на правилното място, което сме му казали:



Генерира се автокод, който много прилича на познатия Create:



Quote

QuoteBG

QuoteEN

Create

Back to List

Унаследяването на модела на първия ред трябва да се отстрани, т.к. не искаме да създаваме нови цитати в тази страница, а да търсим съществуващи.

```
Search.cshtml  X site.css  Create.cshtml  QuotesController.cs  Index.cshtml
Shakesperiada
1  @model Shakesperiada.Models.Quote без наследяване на модел
2
3  <h4>Quote</h4>
4  <hr />
5  <div class="row">
6  <div class="col-md-4">
7  <form asp-action="Search">
8  <div asp-validation-summary="ModelOnly" class="text-danger"></div>
9  <div class="form-group">
10  <label asp-for="QuoteBG" class="control-label"></label>
11  <input asp-for="QuoteBG" class="form-control" />
12  <span asp-validation-for="QuoteBG" class="text-danger"></span>
13  </div>
14  <div class="form-group">
15  <label asp-for="QuoteEN" class="control-label"></label>
16  <input asp-for="QuoteEN" class="form-control" />
17  <span asp-validation-for="QuoteEN" class="text-danger"></span>
18  </div>
19  <div class="form-group">
20  <input type="submit" value="Create" class="btn btn-primary" />
21  </div>
22  </form>
23  </div>
24  </div>
25
26  <div>
27  <a asp-action="Index">Back to List</a>
28  </div>
29
30  @section Scripts {
31  @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
32  }
99 %  No issues found
```

Веднага след изтриването, вече принадлежащите му QuoteBG се подчертават в червено, защото вече не се разпознават.

Search.cshtml | site.css | Create.cshtml | QuotesController.cs | Index.cshtml

Shakesperiada

```

1 <h4>Търси</h4>
2 <hr />
3 <div class="row">
4   <div class="col-md-4">
5     <form asp-action="Search">
6       <div asp-validation-summary="ModelOnly" class="text-danger"></div>
7       <div class="form-group">
8         <label asp-for="QuoteBG" class="control-label">Цитат</label>
9         <input name="QuoteBG" class="form-control" />
10      </div>
11      <div class="form-group">
12        <input type="submit" value="Търси" class="btn btn-primary" />
13      </div>
14    </form>
15  </div>
16 </div>
17
18 <div>
19   <a asp-action="Index">Обратно в списъка</a>
20 </div>
21

```

asp-for = динамична операция

for = статична

name = статична

99 % No issues found

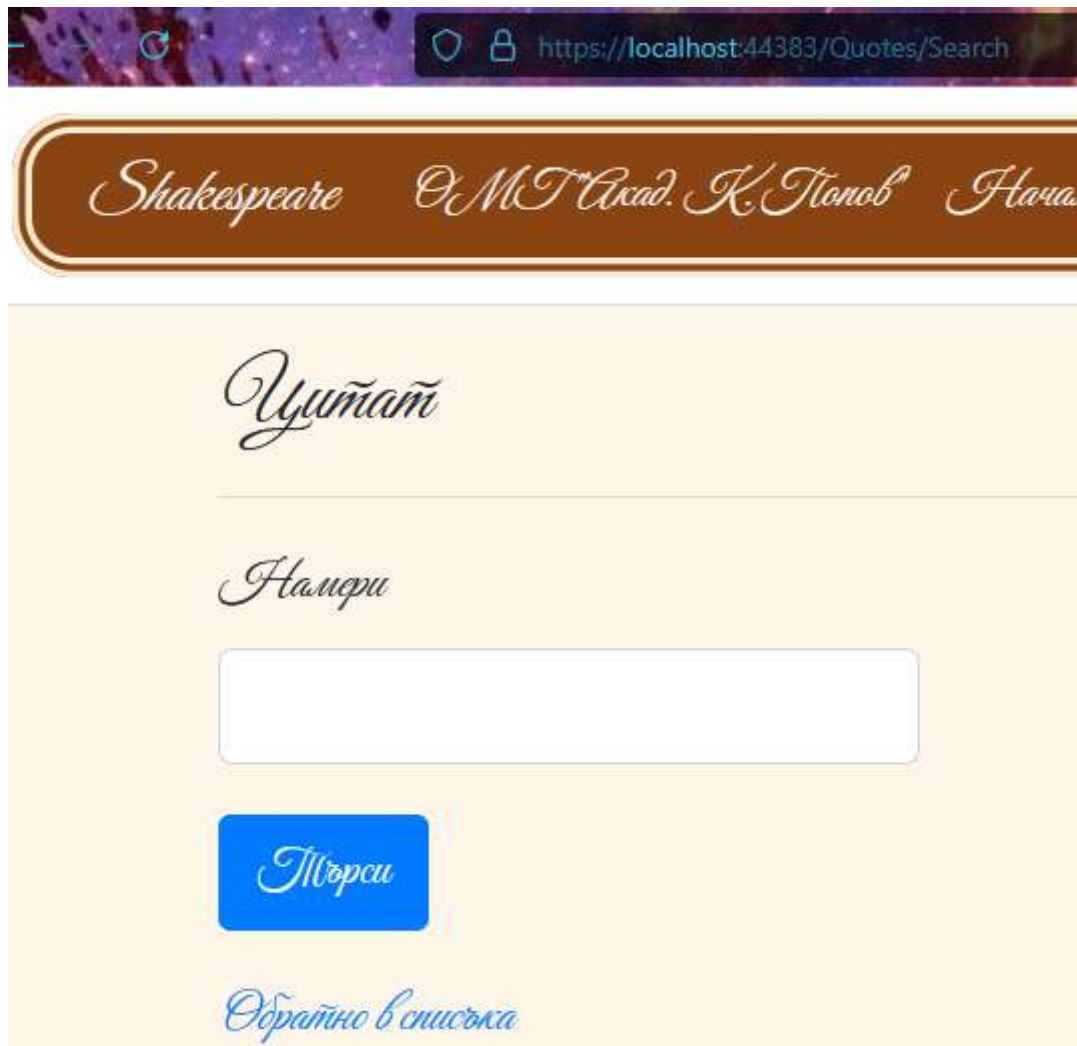
Error List

Entire Solution 2 Errors 0 of 2 Warnings 0 Messages Build + IntelliSense

Code	Description	Project	File	Line
CS1963	An expression tree may not contain a <u>dynamic operation</u>	Shakesperiada	Search.cshtml.g.cs	102
CS1963	An expression tree may not contain a <u>dynamic operation</u>	Shakesperiada	Search.cshtml.g.cs	126

С малко редакции за преводите:

```
Search.cshtml  site.css  Create.cshtml  QuotesController.cs  Index.cshtml
C# Shakespeara
1  <h4>Цитати</h4>
2  <hr />
3  <div class="row">
4  <div class="col-md-4">
5  <form asp-action="Search">
6  <div asp-validation-summary="ModelOnly" class="text-danger"></div>
7  <div class="form-group">
8  <label for="QuoteBG" class="control-label">Намери</label>
9  <input name="QuoteBG" class="form-control" />
10 </div>
11 <div class="form-group">
12 <input type="submit" value="Търси" class="btn btn-primary" />
13 </div>
14 </form>
15 </div>
16 </div>
17
18 <div>
19 <a asp-action="Index">Обратно в списъка</a>
20 </div>
21
99 %  No issues found
```



Ако се опитаме да търсим, все още резултатът ще бъде неуспешен, защото се налагат няколко големи уточнения. Нека конкретизираме, за да разберем по-добре принципите на действие:

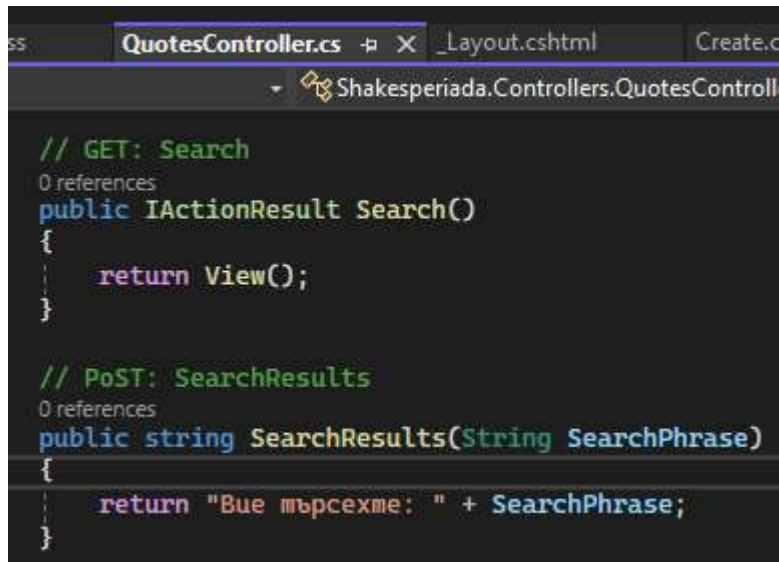
```
<div class="col-md-4">
  <form asp-action="SearchResults">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="form-group">
      <label for="SearchPhrase" class="control-label">Намери</label>
      <input name="SearchPhrase" class="form-control" />
    </div>
    <div class="form-group">
```

```
<h4>Цитат</h4>
<hr />
<div class="row">
  <div class="col-md-4">
    <form asp-action="SearchResults">
      <div asp-validation-summary="ModelOnly" class="text-danger"></div>
      <div class="form-group">
        <label for="SearchPhrase" class="control-label">Намери</label>
```

```

        <input name="SearchPhrase" class="form-control" />
    </div>
    <div class="form-group">
        <input type="submit" value="Търси" class="btn btn-primary" />
    </div>
</form>
</div>
</div>
<div>
    <a asp-action="Index">Обратно в списъка</a>
</div>

```



```

// GET: Search
0 references
public IActionResult Search()
{
    return View();
}

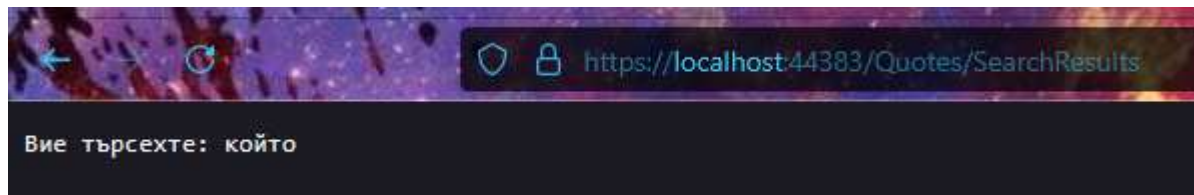
// POST: SearchResults
0 references
public string SearchResults(String SearchPhrase)
{
    return "Вие търсехте: " + SearchPhrase;
}

```

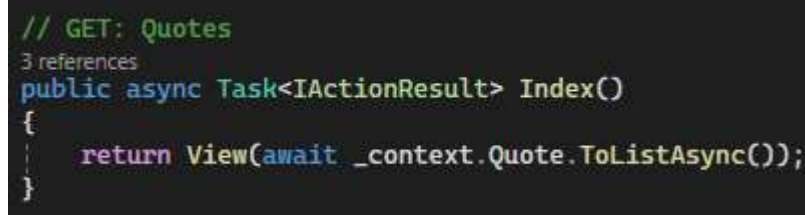
Подаваме и командата за резултата в контролера, като първо тестваме прихващането на фразата като обикновен string, вместо връщане на цяла страница с резултати.

IActionResult е по-сложен тип от string и връща View.

string връща string.



Нека погледнем как изглежда кода, който връща всички резултати:



```

// GET: Quotes
3 references
public async Task

```

```
return View(await _context.Quotes.ToListAsync());
```

По същия начин, този код ще ни върне всички резултати от списъка, все едно нищо не търсим, макар да подаваме входните данни за фразата:

```
QuotesController.cs* X _Layout.cshtml Create.cshtml Index.cshtml
Shakesperiada.Controllers.QuotesController Details(int?)

// PoST: SearchResults

0 references
public async Task<IActionResult> SearchResults(string SearchPhrase)
{
    return View("Index", await _context.Quote.ToListAsync());
}
```

Това ще изкара всички резултати в базата данни, но ние искаме филтрирани резултати. Каква е разликата между сортиране и филтриране?

Имаме готов филтър:

Where(j => j.QuoteBG.Contains(SearchPhrase))

```
QuotesController.cs* X _Layout.cshtml Create.cshtml Index.cshtml
Shakesperiada.Controllers.QuotesController SearchResults(string SearchPhrase)

// PoST: SearchResults

0 references
public async Task<IActionResult> SearchResults(string SearchPhrase)
{
    return View("Index", await _context.Quote.Where(j => j.QuoteBG.Contains(SearchPhrase)).ToListAsync());
}
```

```
return View("Index", await _context.Quote.Where(j =>
j.QuoteBG.Contains(SearchPhrase)).ToListAsync());
```

При тест с думата „който“ изкарва 2 резултата, т.е. филтърът работи:



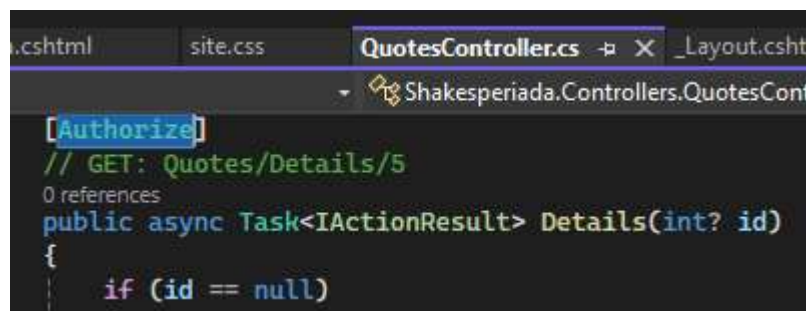
Искаме оригинала на английски да се вижда само при детайли, но не и в index, затова изтриваме неговите полета:

```
Index.cshtml  Search.cshtml  site.css  QuotesController.cs  _Layout.cshtml  Create.cshtml
C# Shakesperiada
9  <p>
10  <a asp-action="Create">Създай нов</a>
11  </p>
12  <table class="table">
13  <thead>
14  <tr>
15  <th>
16  Превод на български:
17  </th>
18  </tr>
19  </thead>
20  <tbody>
21  @foreach (var item in Model) {
22  <tr>
23  <td>
24  @Html.DisplayFor(modelItem => item.QuoteBG)
25  </td>
26  <td>
27  <a asp-action="Edit" asp-route-id="@item.Id">Редактирай</a> |
28  <a asp-action="Details" asp-route-id="@item.Id">Детайли</a> |
29  <a asp-action="Delete" asp-route-id="@item.Id">Изтрий</a>
30  </td>
31  </tr>
```



Трябва да скрием възможността за създаване на нов цитат само за логнат потребител – това става чрез декоратор [Authorize] пред GET и PoST.

Необходима библиотека: `using Microsoft.AspNetCore.Authorization;`



```
.cshtml  site.css  QuotesController.cs  _Layout.cshtml
Shakesperiada.Controllers.QuotesCont

[Authorize]
// GET: Quotes/Details/5
0 references
public async Task<IActionResult> Details(int? id)
{
    if (id == null)
```

По същия начин авторизираме всичките //Get и //PoST на Create, Edit, Delete.

Log in

Use a local account to log in.

Use another service to log in.

Email

test@test.bg

Password

••••••

☐ Remember me?

Log in

[Forgot your password?](#)

[Register as a new user](#)

There are no external authentication services configured.
See this article about setting up this ASP.NET
application to support logging in via external services.